

Research Paper Topics & Experimental Designs for ScholarFlow

This document outlines **15 research topics** with complete experimental designs. Each includes hypothesis, datasets, metrics, visualization suggestions, and implementation guidance specific to your ScholarFlow codebase.

Theme A: Architecture & Workflow (Agentic Systems)

1. Agentic vs. Static RAG Pipelines ■

Title: *Evaluating the Efficacy of Agentic Workflows in Academic Literature Review Systems*

Why this is a paper: The industry is moving from simple "RAG" to "Agentic RAG". Quantitatively proving that multi-step agent loops outperform single-pass chains addresses a critical gap in RAG literature. This has practical implications for cost-benefit analysis in production systems.

Hypothesis: A recursive agentic loop (Router → Search → Rank → Refine) outperforms a linear RAG chain in answer accuracy and completeness for complex, multi-hop research queries.

Experimental Design: * **Control Group (Baseline):** Build a simplified linear chain: `python query → vector_store.search_similar() → answer_generator.generate_answer()` Skip router, ranker, and all feedback loops. * **Experimental Group:** Full `nodes.py` LangGraph workflow with all nodes active. * **Dataset:** Create 50 research questions in 3 categories: * **Simple (n=15):** "What is CRISPR?" (Single fact lookup) * **Medium (n=20):** "How does CRISPR compare to TALEN?" (Two-hop reasoning) * **Complex (n=15):** "What are the ethical implications of CRISPR in agriculture vs. medicine?" (Multi-faceted synthesis) * **Gold Standard:** Have 2 PhD students manually rate each answer 1-5 on: * Accuracy * Completeness * Citation quality

Metrics: * **Primary:** Mean rating score (1-5 scale) * **Secondary:** * Precision@5 (% of top 5 retrieved papers that are relevant) * Hallucination Rate (% of generated claims not supported by retrieved papers - use LLM-as-Judge) * Token Cost (Total tokens consumed per query)

Visualization: * **Grouped Bar Chart:** X-axis = Query Complexity (Simple/Medium/Complex), Y-axis = Mean Score, Two bars (Baseline vs. Agentic) * **Cost-Quality Scatter:** X-axis = Total Tokens, Y-axis = Quality Score, Color by system type

Implementation Steps: 1. Create baseline script `experiments/baseline_rag.py` 2. Collect 50 queries in `experiments/datasets/query_set_1.json` 3. Run both systems, log to

2. Architectural Evolution: Script vs. Chain vs. Graph ■■

Title: *From Linear Chains to Cyclic Graphs: Evaluating Resilience and Error Recovery in Autonomous Research Agents*

Why this is a paper: This directly addresses the engineering question: "Is the complexity of LangGraph worth it vs. simpler LangChain?" Few papers empirically measure *resilience* (ability to recover from failures) as a metric.

Hypothesis: Cyclic graphs (LangGraph) recover from errors (empty search results, API failures) significantly better than linear chains (LangChain) or scripts.

Experimental Design: * **3 Implementations:** 1. **Script:** Direct Python `if/else` with API calls 2. **LangChain:** `SequentialChain` (Search → Generate) 3. **LangGraph:** Your current `nodes.py` with conditional edges * **Failure Injection:** Run 30 queries, but: * Queries 1-10: Normal operation * Queries 11-20: Force `search_node` to return empty results (simulate API downtime) * Queries 21-30: Inject malformed paper metadata (missing abstracts)

Metrics: * **Recovery Rate (%):** Did the system produce *any* answer (even if low quality)? * **Task Completion Rate (%):** Did the system produce a *good* answer (score ≥ 3)? * **Mean Iterations to Success:** How many loops before convergence?

Visualization: * **Success Rate by Failure Type:** Stacked bar chart showing % recovered vs. % crashed * **State Transition Diagram:** Visual comparison - straight line (Chain) vs. cyclic graph (LangGraph)

Expected Result: LangGraph should have ~80%+ recovery rate due to `refine_query_node` retry logic, while Linear Chain crashes immediately.

3. Recursive Critique Efficacy (Self-Correction)

Title: *The Impact of Automated Self-Correction Loops on Academic Writing Quality*

Why this is a paper: LLM hallucinations are a major concern. "Self-correction" via critique loops is a hot mitigation strategy, but few studies measure *how much* it helps in domain-specific tasks like academic writing.

Hypothesis: A Writer → Reviewer → Writer loop (max 2 revisions) produces measurably higher quality academic drafts than single-pass generation.

Experimental Design: * **Variable A (Control):** `writer_node` only, no `reviewer_node` * **Variable B (Experiment):** Full review loop with `reviewer_node` active * **Task:** Generate 20 "Introduction" sections for different research topics * **Evaluation:** Blind A/B test with 3 reviewers (grad students or professors) * Each reviewer sees pairs (A vs. B) in random order * Rates each on: Coherence (1-5), Citation Accuracy (1-5), Flow (1-5)

Metrics: * **Primary:** Win/Loss ratio (How often does B beat A?) * **Secondary:** * Average revision count before approval * Fabricated citation rate (% of citations to non-existent papers)

Visualization: * **Win Rate:** Simple bar chart showing % wins for Single-Pass vs. Revised * **Iteration Distribution:** Histogram of "How many revisions were needed?" (0, 1, 2, 2+)

4. Latency vs. Quality Trade-offs ■

Title: *Cost-Benefit Analysis of Multi-Step Reasoning in Retrieval-Augmented Generation*

Why this is a paper: Agentic systems are expensive (multiple LLM calls). Engineering papers that identify the "sweet spot" (optimal # of iterations before diminishing returns) are valuable for production deployment decisions.

Hypothesis: There exists an optimal iteration count (likely 2-3) where answer quality plateaus but latency continues increasing linearly.

Experimental Design: * **Variable:** Max iterations setting in LangGraph (1, 2, 3, 5, 10) * **Constant:** Same 30 complex queries for all runs * **Measurement:** For each configuration, record: * Total wall-clock time (s) * Total LLM tokens consumed * Quality score (human-rated 1-5)

Metrics: * **Quality Score per Second of Latency** * **Quality Score per 1000 Tokens** * **Pareto Frontier:** Identify configurations that are NOT dominated (i.e., no other config is both faster AND better)

Visualization: * **Elbow Plot:** X-axis = Iteration count, Y-axis = Quality score with error bars. Mark the "knee" of the curve. * **Cost-Quality Frontier:** Scatter plot with Time(s) on X, Quality on Y. Pareto-optimal points highlighted.

Actionable Insight: "For ScholarFlow, 2 iterations is optimal: 3+ iterations add 5s latency for only +0.1 quality improvement."

Theme B: Retrieval & Semantics

5. Query Expansion Efficacy ■

Title: *Impact of LLM-Driven Query Expansion on Retrieval Recall in Specialized Domains*

Why this is a paper: User queries are often vague ("rag vs llm"). Query expansion via LLMs is common but rarely rigorously evaluated. This measures the "value add" of your `QueryAnalyzer`.

Hypothesis: LLM-expanded queries retrieve more relevant papers than raw user inputs, especially for vague/colloquial queries.

Experimental Design: * **Variable A:** Embed the raw user query directly * **Variable B:** Use `QueryAnalyzer.analyze_query()` output as the search query * **Dataset:** 25 queries ranging from technical ("transformer attention mechanisms") to vague ("how does AI work") * **Gold Standard:** Manually curate a "ground truth" set of 5-10 relevant papers per query

Metrics: * **Recall@10:** What % of gold standard papers are in the top 10 results? * **Unique Papers Found:** How many papers does Expanded retrieve that Raw doesn't?

Visualization: * **Venn Diagram:** Overlap between papers retrieved by Raw vs. Expanded queries * **Recall by Query Type:** Bar chart comparing Recall@10 for Technical vs. Vague queries

6. Embedding Model Sensitivity ■■■

Title: *Benchmarking General vs. Domain-Specific Embeddings for Scientific Literature Retrieval*

Why this is a paper: `all-MiniLM-L6-v2` is the default, but `allenai/specter` was trained on scientific papers. Comparing them empirically fills a gap for researchers building academic tools.

Hypothesis: Domain-specific embeddings (`specter`) outperform general-purpose embeddings (`all-MiniLM`) for scientific paper retrieval tasks.

Experimental Design: * **Models to Compare:** 1. `all-MiniLM-L6-v2` (current) 2. `allenai/specter` (scientific) 3. `text-embedding-3-large` (OpenAI, high-dimensional) * **Procedure:** * Index same 100 papers from arXiv with each model * Run 20 domain-specific queries (biology, CS, physics) * **Gold Standard:** Use paper citation graphs (papers that cite each other are relevant)

Metrics: * **Mean Reciprocal Rank (MRR):** Average of $1/\text{rank}$ of first relevant result * **nDCG@10:** Normalized Discounted Cumulative Gain

Visualization: * **MRR Comparison:** Bar chart per model * **Per-Domain Performance:** Heatmap showing MRR for each (Model $\times$ Domain) combination

Implementation: Swap `self.embedding_model` in `vector_store.py` line 19.

7. Chunking Strategies

Title: *Optimizing Context Window Utilization through Adaptive Chunking Strategies in RAG Systems*

Why this is a paper: Chunking is a critical yet under-studied preprocessing step. Most papers use fixed 512-token chunks without justification.

Hypothesis: Semantic chunking (split by section headers or paragraphs) preserves context better than fixed-size chunking.

Experimental Design: * **Strategies to Compare:** 1. Fixed 256 tokens 2. Fixed 512 tokens 3. Fixed 1024 tokens 4. Semantic (by paragraph breaks) 5. Recursive (split until each chunk fits, preferring semantic boundaries) * **Test:** 10 scientific papers, 5 questions per paper requiring specific facts

Metrics: * **Fact Retrieval Accuracy (%):** Is the answer in the retrieved chunk? * **Answer Quality Score:** LLM-generated answer rated 1-5

Visualization: * **Accuracy by Strategy:** Bar chart * **Chunk Size Distribution:** Box plot showing chunk length variance per strategy

8. Lost in the Middle Phenomenon ■

Title: *Context Position Bias in Long-Form Academic Synthesis: A "Needle in Haystack" Study*

Why this is a paper: Recent research shows LLMs suffer from "lost in the middle" bias. Testing this in your domain (academic RAG) is novel.

Hypothesis: Facts placed in the middle of the `literature_context` string are cited less frequently than facts at the start/end.

Experimental Design: * **Setup:** For each query, construct context with 10 paper excerpts * **Injection:** Insert a fake but plausible fact (the "needle") at positions: 10%, 30%, 50%, 70%, 90% through the context * **Measure:** Does the generated answer mention the needle? * **Repeat:** 50 trials with different needles

Metrics: * **Retrieval Rate by Position (%):** What % of needles were mentioned?

Visualization: * **Position Bias Curve:** Line graph, X-axis = Position (0-100%), Y-axis = Retrieval Rate

Expected Result: U-shaped curve (high at 0% and 100%, dip at 50%)

Theme C: Generation & Synthesis

9. Context-Aware Generation (Section Weighting) ■■

Title: *Section-Aware Dynamic Context Weighting in Automated Literature Review Systems*

Why this is a paper: Your `writer_node` uses `get_context_weights()` to prioritize "Student Work" for Methods and "Literature" for Intro. This is a novel architectural choice worth validating.

Hypothesis: Section-aware weighting reduces hallucinations and improves relevance compared to uniform context mixing.

Experimental Design: * **Control:** Flat context (all sources weighted equally) * **Experiment:** Your current dynamic weighting * **Test Cases:** Generate 3 sections (Intro, Methods, Results) × 10 topics = 30 outputs * **Evaluation:** Count: * How many times "Methods" section cites lab assets (should be high) * How many times "Intro" section cites external papers (should be high) * "Hallucinated Methods" (methods not in the student's lab data)

Metrics: * **Source Type Distribution Heatmap:** Rows = Sections, Columns = (External Papers, Lab Data, Other), Values = % of citations

Visualization: * **Before/After Heatmap:** Side-by-side showing context weights

10. Multi-Modal RAG (Visual Assets) ■■

Title: *Bridging Modalities: Integrating Visual Laboratory Data into Textual Research Workflows via Vision-Language Models*

Why this is a paper: Most RAG systems ignore images. Your `lab_analyst_node` uses Gemini Vision to extract data from charts—this is cutting-edge.

Hypothesis: Including AI-generated descriptions of experimental figures increases the factual accuracy and numerical precision of auto-generated reports.

Experimental Design: * **Control:** Text-only (ignore all images) * **Experiment:** Text + Vision (`lab_analyst.py` active) * **Test Set:** 15 research topics with associated lab plots/graphs * **Task:** "Write a Results section describing the trends in Figure X" * **Ground Truth:** Human-annotated list of key numerical values visible in each graph

Metrics: * **Numerical Accuracy:** % of numbers in the graph correctly mentioned in text ($\pm 5\%$ tolerance) * **Trend Accuracy:** Binary yes/no - is the asserted trend (increasing/decreasing) correct?

Visualization: * **Accuracy Comparison Table:** Control vs. Experiment * **Example Gallery:** Show side-by-side (Graph → AI Description → Generated Text)

11. Hallucination Mitigation via Structured Output ■

Title: *Automated Citation Verification: Reducing Hallucinations through Structured Output Constraints in Generative Research Assistants*

Why this is a paper: Trust is crucial for AI adoption in academia. Structured JSON output (your current approach) vs. free-form text is an under-explored mitigation strategy.

Hypothesis: Forcing the LLM to output structured JSON (with explicit "summary" and "key_points" fields) reduces fabricated citations compared to free-form paragraph generation.

Experimental Design: * **Prompt A:** "Write a paragraph summarizing these papers." * **Prompt B:** "Output JSON with 'summary' and 'key_points' fields." (Your current `answer_generator.py`) * **Test:** 30 queries with known paper sets * **Verification:** For each cited author/year, check against the input papers

Metrics: * **Fabrication Rate (%):** % of citations to non-existent papers * **Citation Precision:** % of generated citations that are accurate

Visualization: * **Error Rate Bar Chart:** Prompt A vs. Prompt B

12. Information Density Analysis

Title: *Summarization vs. Structured Extraction: Evaluating Information Preservation in Research Synthesis Tasks*

Why this is a paper: Compression always loses information. Quantifying *how much* is lost helps users understand trade-offs.

Hypothesis: Structured extraction (JSON key points) preserves more "atomic facts" than paragraph summaries at similar token budgets.

Experimental Design: * **Source:** 10 academic abstracts * **Task A:** Summarize to 100 tokens (paragraph) * **Task B:** Extract key facts as JSON (also ~100 tokens) * **Metric:** "Atomic Fact Recall" * Decompose source abstract into atomic facts using GPT-4 * Check how many facts survive in each output format

Metrics: * **Atomic Fact Recall (%):** What % of original facts are present? * **Compression Ratio:** Original tokens / Output tokens

Visualization: * **Recall by Method:** Bar chart

Theme D: User Interaction & System Design

13. Intent Classification Impact

Title: *The Role of Intent Recognition in Adaptive Research Assistance Systems*

Why this is a paper: Personalization is a key UX improvement, but does it measurably affect outcomes? Your `router_node` makes routing decisions—test if they matter.

Hypothesis: Intent-aware routing (Quick Answer vs. Deep Dive) improves user satisfaction compared to a one-size-fits-all approach.

Experimental Design: * **System A:** Router disabled - all queries use same deep workflow * **System B:** Router active * **User Study:** 10 participants × 10 queries each * Mix of quick factual and deep research queries * **Measure:** Time to satisfaction, quality rating

Metrics: * **Time to Answer (s)** * **User Satisfaction (1-5 Likert scale)**

Visualization: * **Satisfaction by Query Type:** Grouped bar chart

14. Prompt Engineering: Zero-Shot vs. Few-Shot

Title: *Few-Shot Learning in Academic Query Understanding: An Empirical Evaluation*

Why this is a paper: Prompt engineering is critical but often ad-hoc. Systematic comparison of zero-shot vs. few-shot is publishable.

Hypothesis: Adding 3 in-prompt examples (few-shot) to `QueryAnalyzer` prompt significantly improves query expansion quality.

Experimental Design: * **Prompt A:** Current zero-shot prompt * **Prompt B:** Same but with 3 exemplars:
`Example 1: "rag vs llm" → "Comparative analysis of Retrieval-Augmented Generation and Large Language Models" ...` * **Evaluation:** Human judges rate the generated search queries for "Specificity" and "Academic Tone"

Metrics: * **Quality Score (1-5)** from human raters * **Inter-Annotator Agreement (Kappa)**

15. Objective vs. Subjective Quality ■

Title: *Do Automated Relevance Scores Predict User Satisfaction? A Study of Metrics Alignment in AI Research Assistants*

Why this is a paper: You have `RankerNode` generating relevance scores. Do these scores actually correlate with what users think is good? This validates your metric.

Hypothesis: Papers with high `relevance_score` from `ranker_node` receive higher human satisfaction ratings.

Experimental Design: * **Data Collection:** For 50 queries, log: * Ranker's relevance score (0-1) * Human rating of final answer quality (1-5) * **Analysis:** Pearson correlation

Metrics: * **Correlation Coefficient (r):** Are they aligned? * **Confusion Matrix:** How often do high-scored papers get low ratings?

Visualization: * **Scatter Plot:** X = Ranker Score, Y = Human Rating, with trend line * **Categorization Matrix:** 2x2 grid (High Score, Low Score x High Rating, Low Rating)

Expected Result: Moderate positive correlation ($r \approx 0.5-0.7$), but some misalignment reveals edge cases worth investigating.

Summary: Topic Selection Guide

Topic	Difficulty	Data Collection	Novelty	Best For
1. Agentic RAG	Medium	Moderate	High	Final thesis
2. Script→Chain→Graph	Medium	Low	Very High	Conference paper
3. Self-Correction	Easy	High (needs humans)	Medium	Quick paper
4. Latency Trade-offs	Easy	Low	Medium	Engineering report

5. Query Expansion	Easy	Low	Medium	Short paper
6. Embedding Models	Medium	Low	High	Strong comparison study
7. Chunking	Easy	Low	Low	Coursework
8. Lost in Middle	Easy	Low	High	Novel finding
9. Section Weighting	Medium	Moderate	Very High	Your novel contribution
10. Multi-Modal	Hard	Moderate	Very High	Top-tier paper
11. Hallucination	Medium	Moderate	High	Trustworthy AI focus
12. Info Density	Easy	Low	Low	Supplementary analysis
13. Intent Routing	Hard	High (user study)	Medium	HCI paper
14. Few-Shot	Easy	Low	Low	Quick ablation study
15. Metric Validation	Medium	High (user study)	Medium	Metrics paper

■ **Top Recommendations for Maximum Impact:** 1. **Topic 2** (Script→Chain→Graph): Novel architectural comparison with clear practical implications 2. **Topic 9** (Section Weighting): Your unique contribution—novel and implementable 3. **Topic 10** (Multi-Modal): Cutting-edge, addresses major RAG gap 4. **Topic 6** (Embeddings): Rigorous empirical study with immediate value to community